

```

1  =====
2  RECAP - Contiki-NG CoAP Exercise 2
3  IoT Lab - Francesca Righetti (UniPi)
4  =====
5
6  OBIETTIVO
7  -----
8  Implementare un CoAP server con una risorsa (res-myhouse) che
9  supporta GET, POST e PUT. La risorsa gestisce un array di stanze.
10
11
12  =====
13  CODICE COMPLETO - res-myhouse.c
14  =====
15
16  #include <stdlib.h>
17  #include <string.h>
18  #include <stdio.h>
19  #include "coap-engine.h"
20
21  #define MAX_ROOMS 7
22  #define ROOM_NAME_LEN 20
23
24  static void res_get_handler(coap_message_t *request, coap_message_t *response,
25                             uint8_t *buffer, uint16_t preferred_size, int32_t *offset);
26  static void res_post_handler(coap_message_t *request, coap_message_t *response,
27                               uint8_t *buffer, uint16_t preferred_size, int32_t *offset);
28  static void res_put_handler(coap_message_t *request, coap_message_t *response,
29                              uint8_t *buffer, uint16_t preferred_size, int32_t *offset);
30
31  RESOURCE(res_myhouse,
32           "title=\"MyHouse: ?room=0..\" POST/PUT name=<name>&value=<value>\";rt=\"Control\"",
33           res_get_handler,
34           res_post_handler,
35           res_put_handler,
36           NULL);
37
38  static char rooms[MAX_ROOMS][ROOM_NAME_LEN] = {
39     "kitchen",
40     "bedroom a",
41     "hall",
42  };
43  static int actual_rooms = 3;
44
45  /* ----- */
46  /* GET */
47  /* ----- */
48  static void res_get_handler(coap_message_t *request, coap_message_t *response,

```

```

49         uint8_t *buffer, uint16_t preferred_size, int32_t *offset) {
50     const char *room_param = NULL;
51     int len = 0;
52
53     if(coap_get_query_variable(request, "room", &room_param)) {
54         int room_index = atoi(room_param);
55         if(room_index >= 1 && room_index <= actual_rooms) {
56             len = snprintf((char *)buffer, preferred_size, "%s, ", rooms[room_index - 1]);
57         } else {
58             coap_set_status_code(response, BAD_REQUEST_4_00);
59             return;
60         }
61     } else {
62         len = 0;
63         for(int i = 0; i < actual_rooms; i++) {
64             len += snprintf((char *)buffer + len, preferred_size - len, "%s, ", rooms[i]);
65             if(len >= preferred_size) break;
66         }
67     }
68
69     coap_set_header_content_format(response, TEXT_PLAIN);
70     coap_set_payload(response, buffer, len);
71 }
72
73 /* ----- */
74 /* POST - aggiunge una stanza (parsa payload raw, NON usa */
75 /* coap_get_post_variable perche' non funziona affidabilmente) */
76 /* ----- */
77 static void res_post_handler(coap_message_t *request, coap_message_t *response,
78                             uint8_t *buffer, uint16_t preferred_size, int32_t *offset) {
79
80     const uint8_t *payload = NULL;
81     int payload_len = coap_get_payload(request, &payload);
82
83     printf("POST payload (%d bytes): %.*s\n", payload_len, payload_len, (char *)payload);
84
85     if(payload_len <= 0 || actual_rooms >= MAX_ROOMS) {
86         coap_set_status_code(response, BAD_REQUEST_4_00);
87         printf("POST failed: payload_len=%d, actual_rooms=%d\n", payload_len, actual_rooms);
88         return;
89     }
90
91     char tmp[64];
92     memset(tmp, 0, sizeof(tmp));
93     memcpy(tmp, payload, payload_len < 63 ? payload_len : 63);
94
95     printf("POST tmp: %s\n", tmp);
96

```

```

97     char *name_val = strstr(tmp, "name=");
98     if(name_val == NULL) {
99         coap_set_status_code(response, BAD_REQUEST_4_00);
100        printf("POST failed: 'name=' not found in payload\n");
101        return;
102    }
103
104    name_val += 5;
105
106    char *end = strchr(name_val, '&');
107    if(end) *end = '\\0';
108
109    printf("POST name_val: %s\n", name_val);
110
111    snprintf(rooms[actual_rooms], ROOM_NAME_LEN, "%s", name_val);
112    actual_rooms++;
113    coap_set_status_code(response, CREATED_2_01);
114    printf("Room added: %s. Total rooms: %d\n", rooms[actual_rooms - 1], actual_rooms);
115 }
116
117 /* ----- */
118 /* PUT - modifica una stanza esistente */
119 /* ----- */
120 static void res_put_handler(coap_message_t *request, coap_message_t *response,
121                             uint8_t *buffer, uint16_t preferred_size, int32_t *offset) {
122     const char *text = NULL;
123     char room_name[ROOM_NAME_LEN];
124     char new_value[ROOM_NAME_LEN];
125     memset(room_name, 0, ROOM_NAME_LEN);
126     memset(new_value, 0, ROOM_NAME_LEN);
127
128     size_t len = coap_get_post_variable(request, "name", &text);
129     if(len > 0 && len < ROOM_NAME_LEN) {
130         memcpy(room_name, text, len);
131     }
132
133     len = coap_get_post_variable(request, "value", &text);
134     if(len > 0 && len < ROOM_NAME_LEN) {
135         memcpy(new_value, text, len);
136     }
137
138     int found = 0;
139     for(int i = 0; i < actual_rooms; i++) {
140         if(strncmp(rooms[i], room_name, ROOM_NAME_LEN) == 0) {
141             snprintf(rooms[i], ROOM_NAME_LEN, "%s", new_value);
142             found = 1;
143             break;
144         }

```

```

145     }
146
147     if(found) {
148         coap_set_status_code(response, CHANGED_2_04);
149     } else {
150         coap_set_status_code(response, NOT_FOUND_4_04);
151     }
152 }
153
154
155 =====
156 PROBLEMI RISCONTRATI E SOLUZIONI
157 =====
158
159 PROBLEMA 1 - coap_get_post_variable restituisce sempre 0
160 -----
161 Sintomo:
162     La POST handler riceveva len=0 da coap_get_post_variable e
163     rispondeva sempre con 4.00 Bad Request.
164
165 Causa:
166     coap_get_post_variable funziona SOLO se il Content-Format
167     del messaggio CoAP e' 47 (application/x-www-form-urlencoded).
168     Se il Content-Format e' assente o diverso, la funzione non
169     trova i parametri e restituisce 0.
170
171 Soluzione adottata:
172     Bypassare coap_get_post_variable e parsare il payload raw
173     con coap_get_payload + strstr("name="), che funziona
174     indipendentemente dal Content-Format.
175
176
177 PROBLEMA 2 - POST con payload vuoto (0 bytes) usando -L 17 e -e
178 -----
179 Sintomo:
180     Con il comando:
181         coap-client -L 17 -m post 'coap://[...]/home' -t 47 -e 'name=bathroom'
182     il serial stampava: "POST payload (0 bytes):"
183     Il payload arrivava completamente vuoto.
184
185 Causa:
186     Il flag -L 17 = -L 1 (NON-confirmable) + -L 16 (block1 transfer
187     per PUT/POST). Il block1 transfer interferisce con il payload
188     quando -e e -m post vengono specificati DOPO l'URL.
189
190 Soluzione:
191     Invertire l'ordine degli argomenti: mettere -t e -e PRIMA di
192     -m post e dell'URL:

```

```
193     coap-client -L 17 -t 47 -e 'name=bathroom' -m post 'coap://[...]/home'
194
195     NOTA: questo e' un bug/comportamento del coap-client di libcoap
196     su macOS. L'ordine degli argomenti cambia il comportamento.
197
198
199     PROBLEMA 3 - Senza -L 17 i messaggi non arrivano al nodo
200     -----
201     Sintomo:
202     Comandi senza -L 17 (es. -L 1 o nessun flag) non producevano
203     alcun output nel serial del nodo, come se il messaggio non
204     arrivasse.
205
206     Causa:
207     Su macOS con questa configurazione di rete (tunnel IPv6 verso
208     Cooja via border router), solo la modalita' -L 17 garantisce
209     che i pacchetti vengano instradati correttamente.
210
211     Soluzione:
212     Usare sempre -L 17 per tutti i comandi, con l'accortezza di
213     mettere -t e -e PRIMA di -m e dell'URL per le POST/PUT.
214
215
216     =====
217     COMANDI FUNZIONANTI (macOS, -L 17)
218     =====
219
220     Sostituire fd00::202:2:2:2 con l'indirizzo IPv6 del proprio nodo.
221
222     GET - tutte le stanze:
223     coap-client -L 17 -m get 'coap://[fd00::202:2:2:2]/home'
224
225     GET - stanza specifica (indice 1-based):
226     coap-client -L 17 -m get 'coap://[fd00::202:2:2:2]/home?room=1'
227     coap-client -L 17 -m get 'coap://[fd00::202:2:2:2]/home?room=2'
228     coap-client -L 17 -m get 'coap://[fd00::202:2:2:2]/home?room=3'
229
230     POST - aggiunge una stanza:
231     coap-client -L 17 -t 47 -e 'name=bathroom' -m post 'coap://[fd00::202:2:2:2]/home'
232
233     IMPORTANTE: -t 47 e -e 'name=...' devono venire PRIMA di -m post e dell'URL.
234
235     PUT - modifica una stanza (nome corrente -> nuovo valore):
236     coap-client -L 17 -t 47 -e 'name=kitchen&value=cucina' -m put 'coap://[fd00::202:2:2:2]/home'
237
238
239     =====
240     NOTE GENERALI SU CONTIKI-NG COAP
```

```
241 =====
242
243 - coap_get_post_variable: funziona solo con Content-Format 47.
244   Se si vuole robustezza, usare coap_get_payload + strstr manuale.
245
246 - coap_get_query_variable: funziona per parametri URL tipo ?room=1,
247   indipendentemente dal Content-Format.
248
249 - Su macOS con coap-client di libcoap l'ordine degli argomenti
250   e' importante, soprattutto con -L 17 e payload POST/PUT.
251
252 - Il debug con printf nel serial di Cooja e' essenziale:
253   stampare payload_len, content_format e il risultato di
254   coap_get_post_variable aiuta a capire cosa arriva realmente
255   al nodo.
256
257 =====
258
```